

Highlights

DaCaF: Bayes-optimal per-metric inference for probabilistic multi-label classification

Vu-Linh Nguyen, Xuan-Truong Hoang, Anh Hoang, Van-Nam Huynh

- Bayes-optimal per-metric inference for any probabilistic multi-label classifier
- Divide-and-conquer plus fusion: exact optima over two whole metric families
- Seven predict-rules, each verified against brute-force enumeration
- pip-installable and Dockerised, one-command reproduction of paper tables

DaCaF: Bayes-optimal per-metric inference for probabilistic multi-label classification

Vu-Linh Nguyen^a, Xuan-Truong Hoang^{b,*}, Anh Hoang^c and Van-Nam Huynh^d

^a*TODO, TODO*

^b*Japan Advanced Institute of Science and Technology (JAIST), Japan*

^c*TODO, TODO*

^d*Japan Advanced Institute of Science and Technology (JAIST), Japan*

ARTICLE INFO

Keywords:

multi-label classification
probabilistic classifier chains
Bayes-optimal inference
evaluation metrics

ABSTRACT

In multi-label classification (MLC), different evaluation metrics call for different Bayes-Optimal Predictions (BOPs), and no single model is optimal for all of them. Probabilistic MLC allows the target metric to be optimized during the prediction phase, but computing the exact BOP is intractable in general, and existing tools are limited—covering only a few metrics, relying on strong assumptions such as label independence, or lacking a reusable, well-tested implementation. We present DaCaF, a Python package implementing the Divide-and-Conquer and Fusion approach that, given any probabilistic model $P(y | x)$, returns the exact BOP for a chosen metric, covering two families of MLC metrics. The method combines a divide-and-conquer step, which partitions the prediction search space into groups to find a local optimum in each, with a fusion step, which estimates the required probabilities from the chain's dependent binary classifiers. The package provides seven per-metric prediction rules, each verified against brute-force enumeration, together with a command-line interface, a library API, and reproducible evaluation scripts, enabling an exact study of the mismatch between target and evaluation metrics.

1. Motivation and significance

Multi-label classification (MLC) is a setting in which each instance can belong to any subset of the labels, with applications ranging from text and image annotation to high-stakes domains such as clinical decision-making. To compare ground-truth label sets and predictions, a variety of evaluation metrics with different characteristics have been proposed. Probabilistic MLC offers unique advantages by allowing the optimization of different metrics during the prediction phase: once a probabilistic classifier estimates the conditional probability of each labeling, the Bayes-Optimal Prediction (BOP) of a target metric—the prediction $\hat{y}$ that maximizes the expected score of that metric—can be sought. As different metrics may call for significantly different BOPs, a mismatch between the target metric, i.e., the metric optimized during the prediction phase, and the evaluation metric, i.e., the metric used to assess the classifier, can negatively impact predictive performance.

However, computing the BOP in a naive way—by searching over the 2^L candidate predictions for L labels—quickly becomes intractable as the number of labels grows. Existing studies are arguably limited in two respects: some focus on only a small number of metrics, such as subset 0/1 accuracy, Hamming accuracy, and F_β (Dembczyński et al., 2010, 2011; Waegeman et al., 2014), yet require a different, metric-specific algorithm for each metric; others rely on strong assumptions, such as label independence (Pillai et al., 2017). As a consequence, no reusable, exact, and broadly applicable

tool exists for finding BOPs across the metrics commonly used in probabilistic MLC.

We present DaCaF, a software package that implements the Divide-and-Conquer and Fusion (DaCaF) approach (Nguyen et al., 2026): given any probabilistic model $P(y | x)$, it returns the exact BOP for a chosen metric, covering two families of commonly used MLC metrics. It proceeds in two steps—a divide-and-conquer strategy followed by a fusion strategy—to find the prediction that is optimal for the selected metric. DaCaF thereby (a) optimizes the metric one actually cares about, (b) enables studying the mismatch between the target and evaluation metrics in an exact way, undisturbed by approximation, and (c) provides a diagnostic that flags metrics whose BOP has a trivial or very restricted structure—useful for assessing the practical value of emerging MLC metrics.

2. Software description

2.1. Software architecture

- Installable Python package `dacaf_mlc` (pip / Docker; console script `dacaf-mlc`).
- Core: `ProbabilisticClassifierChainCustom` (PCC) wraps any scikit-learn base estimator; estimates $P(y | x)$ via a classifier chain.
- Two algorithmic building blocks behind every inference rule:
 - Divide-and-conquer: partition the 2^L predictions into $L+1$ groups by number of relevant labels; best-in-group by sorting labels by a score; global best = best across groups.

*Corresponding author

 hxtruong@jaist.ac.jp (X. Hoang)
ORCID(s):

Table 1
Code metadata

Nr.	Code metadata description
C1	Current code version: v1.1.0
C2	Permanent link to code/repository: https://github.com/hxtruong6/inference_probabilistic_mlc Zenodo: https://doi.org/10.5281/zenodo.20572638
C3	Permanent link to Reproducible Capsule: https://codeocean.com/capsule/1580907/tree
C4	Legal Code License: MIT
C5	Code versioning system used: git
C6	Software code languages, tools, services used: Python (≥ 3.10); NumPy, SciPy, scikit-learn, pandas, joblib; optional PyTorch / torchvision / torchxrayvision ([image] extra)
C7	Compilation requirements, operating environments, dependencies: <code>pip install -e . (core) or ".[image]"</code> ; Linux/macOS; Docker image provided
C8	Link to developer documentation/manual: README.md, CONVENTIONS.md, docs/
C9	Support email for questions: hxtruong@jaist.ac.jp

- Fusion: scores need marginal/pairwise probabilities, estimated by fusing the chain's dependent binary classifiers (ancestral sampling).

- Supporting modules: `evaluation_metrics.py`, `metrics_registry.py`, `pipeline.py` (train / k-fold / run_single), `evaluate.py` (CLI), `arff_dataset.py` (MULAN ARFF + 10-fold CV), `datasets.py`, `chest_xray_dataset/` (NIH feature extractor, [image] extra).

- Vendored `skmultiflow` `ClassifierChain` base for a self-contained install.

2.2. Software functionalities

- Seven per-metric Bayes-optimal rules: `predict_fmeasure`, `predict_hamming`, `predict_markedness`, `predict_precision`, `predict_npv`, `predict_recall`, `predict_subset`.
- Each returns the prediction maximising the expected value of its target metric; conventions in `CONVENTIONS.md`.
- Covers two whole metric families at once; flags when a metric's optimum is *trivial* (a metric-choice diagnostic).
- Batched predictor: one `predict_proba` per chain level instead of $N \cdot L \cdot 2^L$; verified numerically equal to brute-force enumeration.
- Reproducible evaluation: CLI + scripts (`reproduce_tabular.sh`, Slurm) over MULAN tabular datasets and an NIH ChestX-ray14 image dataset.
- Correctness harness: every rule checked against brute-force expected-metric enumeration on small cases; CI on Python 3.10/3.12.

3. Illustrative examples

- Minimal library usage (fit once, query any metric's BOP):

```
from sklearn.linear_model import LogisticRegression
from dacaf_mlc.probability_classifier_chains \
    import ProbabilisticClassifierChainCustom
from dacaf_mlc.evaluation_metrics import EvaluationMetrics as EM

pcc = ProbabilisticClassifierChainCustom(
    LogisticRegression(max_iter=10_000))
pcc.fit(X_train, Y_train) # Y: (n, L) binary
y_f1 = pcc.predict_fmeasure(X_test, beta=1) # BOP for F1
y_ham = pcc.predict_hamming(X_test) # BOP for Hamming
print(EM.f_beta(Y_test, y_f1), EM.hamming(Y_test, y_ham))
```

- Headline experiment (mismatch hurts): Table 2, rows = metric optimised for, columns = metric evaluated; bold diagonal = column maximum.
- One command reproduces it: `make reproduce (CHD-49, fastest dataset)`.
- The chest-xray path scales the same API to real DenseNet/ResNet features.

4. Impact

- Enables an exact research question: how much does optimising the wrong metric cost, measured without approximation blurring the picture?
- Model-agnostic: works with *any* probabilistic MLC model; a drop-in BOP layer on top of existing $P(y | x)$ estimators.
- Breadth: one tool covers two whole metric families; the trivial-optimum diagnostic guides metric selection.
- Reuse beyond the paper: clinical/image MLC (NIH ChestX-ray14 demo), benchmarking, and teaching Bayes-optimal decision-making in MLC.
- Trust: closed-form rules verified against brute-force enumeration; CI keeps it reproducible.
- Low barrier: pip-installable, Dockerised, scripted reproduction, MULAN datasets bundled; a newcomer reproduces a paper table in minutes.

Table 2

CHD-49, PCC + logistic regression, mean over 5 seeds $\times$ 10-fold CV (% , higher is better). Read each column: the bold diagonal entry (optimise the metric you evaluate) is the maximum of its column. NPV and Recall collapse to the predict-all trivial optimum.

Target $\downarrow$ / Eval $\rightarrow$	F_1	Hamming	Markedness	NPV	Precision	Recall	Subset
Predict F_1	67.07	69.29	71.91	81.09	62.74	79.22	15.06
Predict Hamming	63.92	70.78	71.27	75.74	66.62	66.93	18.05
Predict Markedness	34.20	63.70	76.96	71.10	33.37	40.45	8.39
Predict NPV	58.35	43.01	71.50	100.00	43.01	99.46	0.00
Predict Precision	40.54	64.83	68.31	63.10	73.52	29.10	3.10
Predict Recall	58.35	43.01	71.50	100.00	43.01	99.46	0.00
Predict Subset	63.96	69.37	70.35	76.47	64.15	69.79	18.92

- Extensibility: registry pattern to add metrics/datasets; vendored base keeps installs self-contained.
- Who benefits: MLC researchers, applied ML practitioners choosing a deployment metric, and educators.

5. Conclusions

- DaCaF turns a proved theoretical recipe into a reusable, verified, exact per-metric inference tool for probabilistic MLC.
- Future: more base models, more metric families, optional approximate fast paths.

Declaration of competing interest

The authors declare no competing interests.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

TODO statement (or remove if not applicable).

Acknowledgements

References

- V.-L. Nguyen, X.-T. Hoang, A. Hoang, V.-N. Huynh, Probabilistic multi-label classification via a divide-and-conquer and fusion approach, *Information Fusion* (2026), Article 104517. <https://doi.org/10.1016/j.inffus.2026.104517>.
- X.-T. Hoang, V.-L. Nguyen, A. Hoang, V.-N. Huynh, DaCaF: probabilistic multi-label classification (v1.1.0) [Computer software], Zenodo (2026). <https://doi.org/10.5281/zenodo.20572638>.
- K. Dembczyński, W. Cheng, E. Hüllermeier, Bayes optimal multilabel classification via probabilistic classifier chains, *ICML* 2010.
- K. Dembczyński, W. Waegeman, W. Cheng, E. Hüllermeier, An exact algorithm for F-measure maximization, *NeurIPS* 2011.
- W. Waegeman, K. Dembczyński, A. Jachnik, W. Cheng, E. Hüllermeier, On the Bayes-optimality of F-measure maximizers, *J. Mach. Learn. Res.* 15 (2014) 3513–3568.
- I. Pillai, G. Fumera, F. Roli, Designing multi-label classifiers that maximize F measures: state of the art, *Pattern Recognition* 61 (2017) 394–404.